INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And Exploitation Techniques(Security Misconfiguration)

Student Name: Abubakari-Sadique Hamidu
Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

Title: Lab 8: Broken Authentication

Objective

The objective of this lab is to:

- Understand broken authentication vulnerabilities
- Demonstrate insecure password handling
- Fix the vulnerability using password hashing and session management

Task 1: Exploiting Broken Authentication

Description

In this task, a vulnerable login system was created using plain-text password storage and no session management.
This makes the system easy to attack.

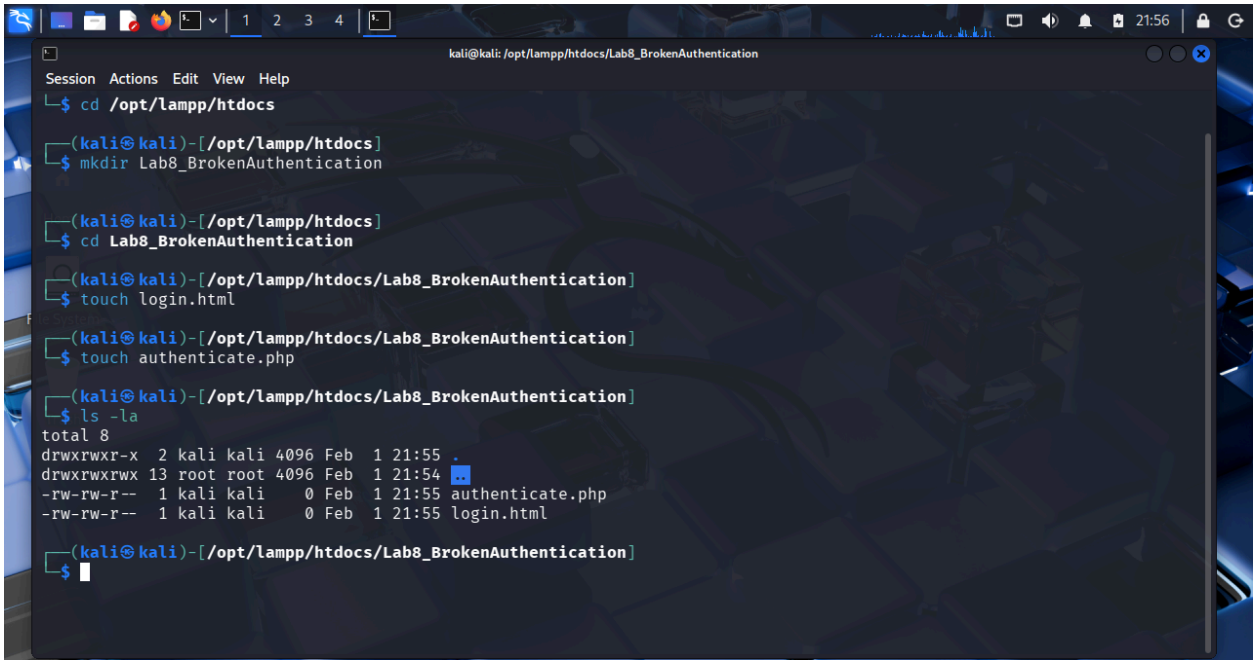
Steps Performed

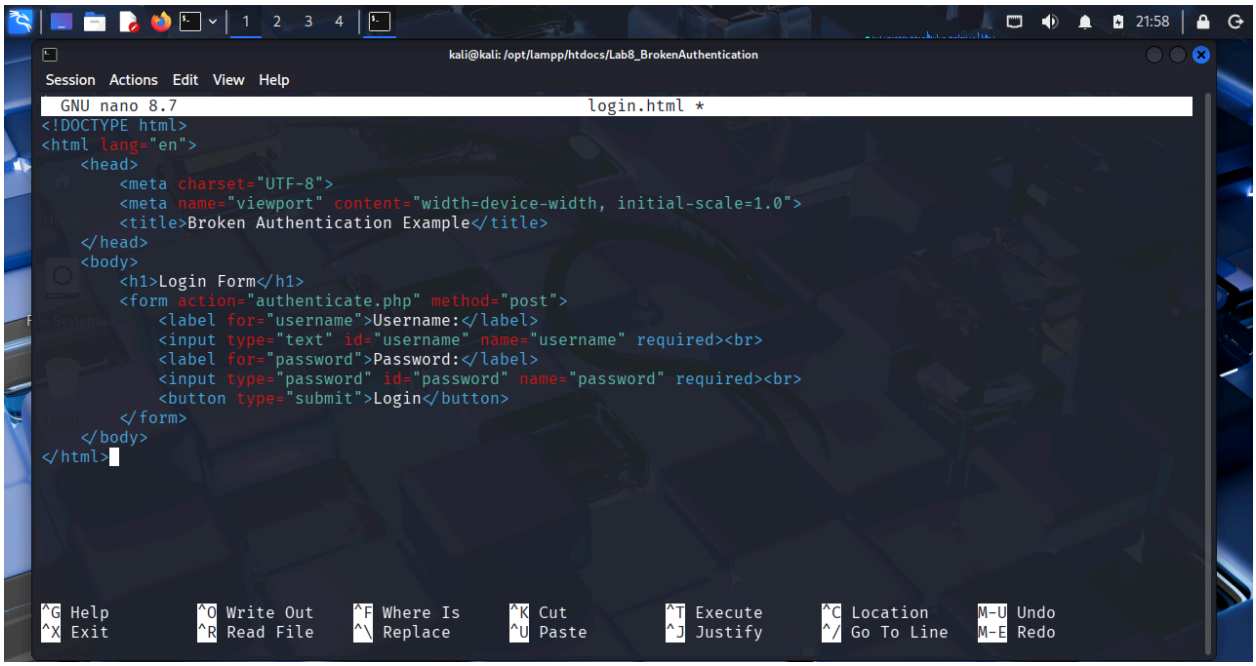
- Created Lab8 folder
- Created a login form (`login.html`)
- Created a PHP authentication file (`authenticate.php`)
- Stored the password in plain text
- Logged in using visible credentials

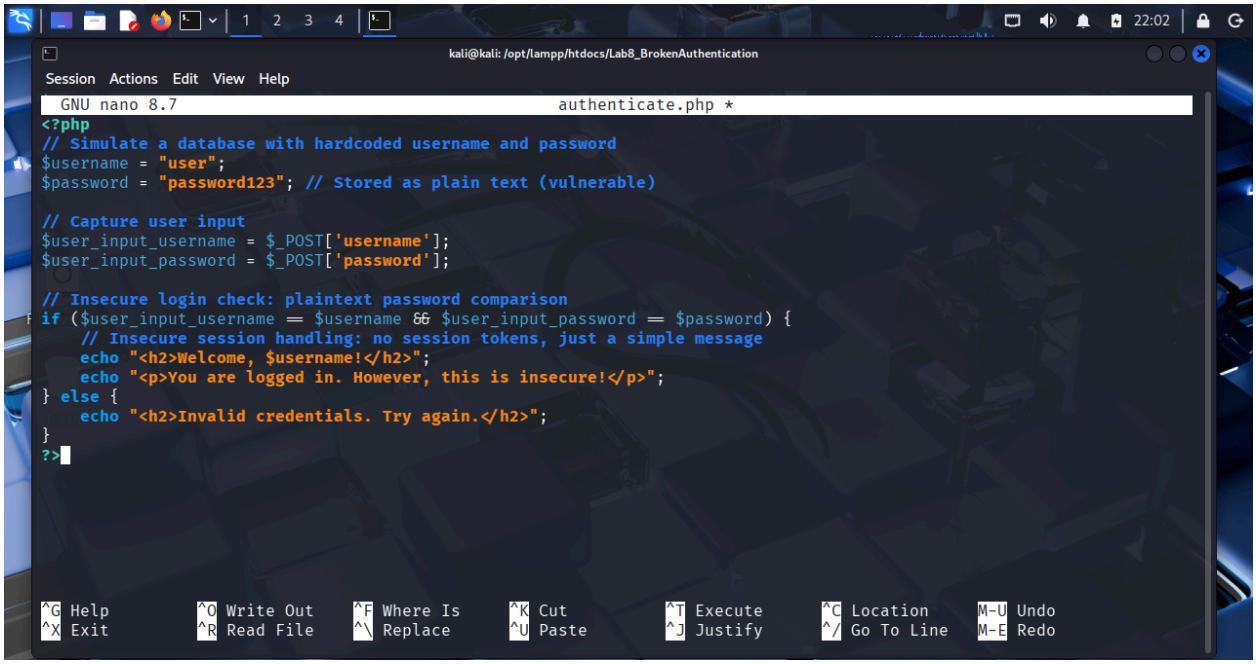
Observation

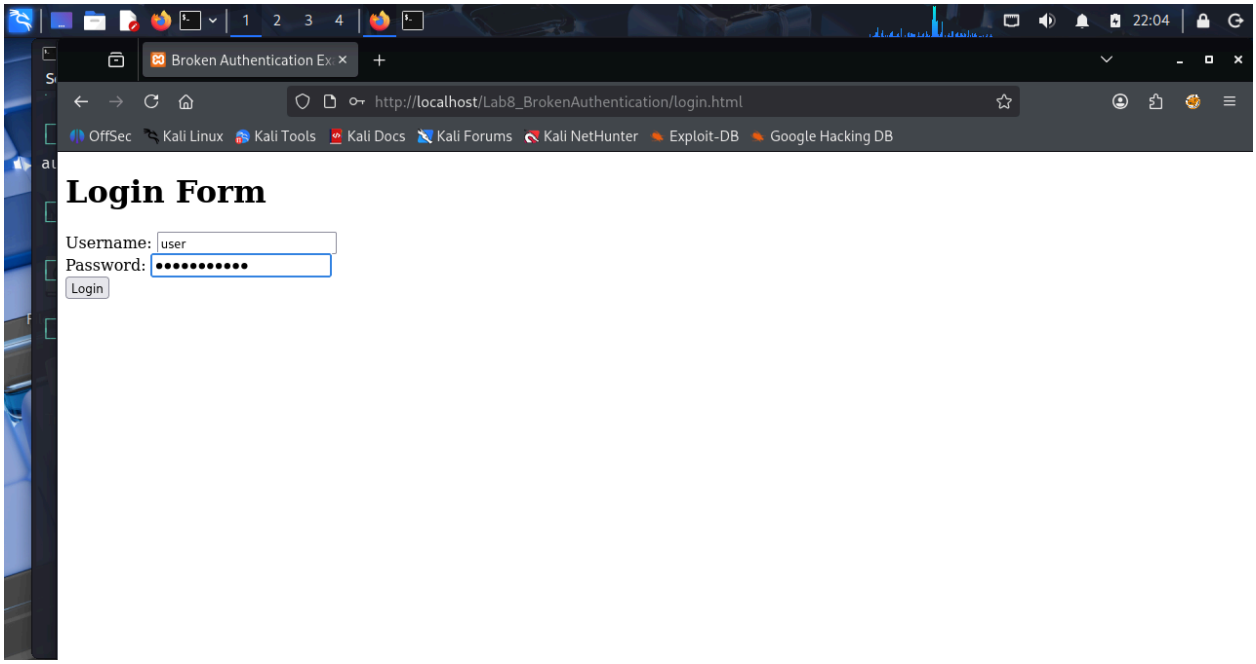
- The password was readable in the source code
- Authentication was done using direct string comparison
- No session was created after login

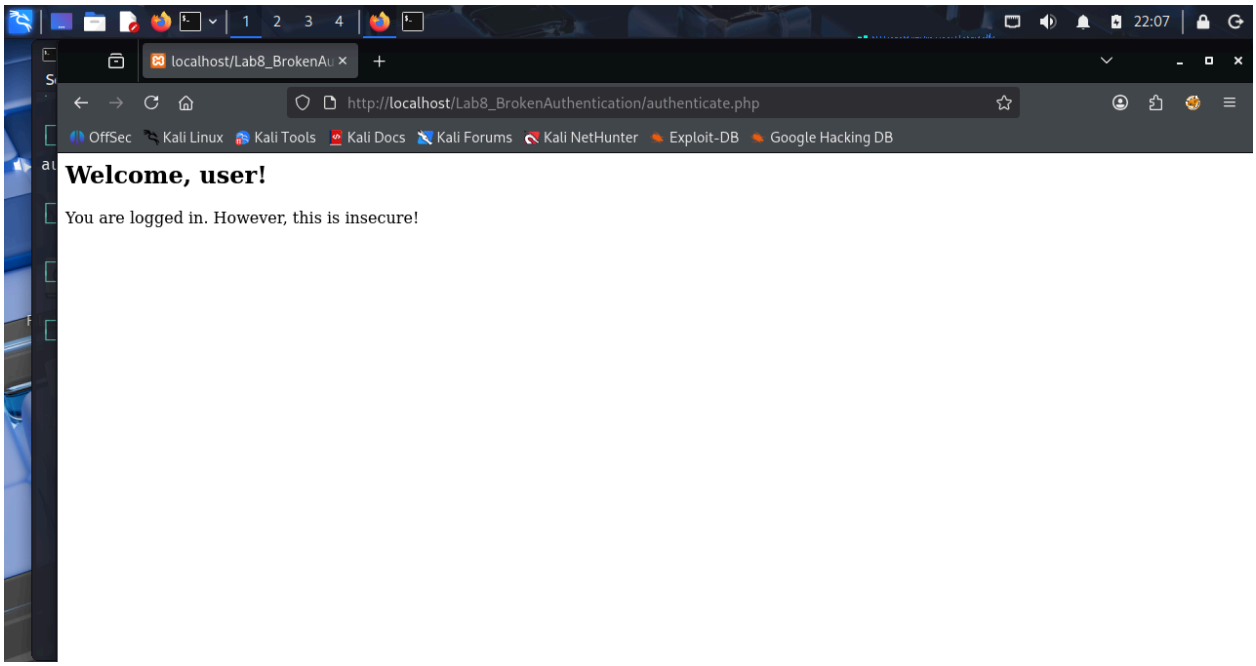
This confirms the application is vulnerable to broken authentication.


kali@kali: /opt/lampp/htdocs/Lab8_BrokenAuthentication
Session Actions Edit View Help
cd /opt/lampp/htdocs
mkdir Lab8_BrokenAuthentication
cd Lab8_BrokenAuthentication
touch login.html
touch authenticate.php
ls -la
total 8
drwxrwxr-x 2 kali kali 4096 Feb 1 21:55 .
drwxrwxrwx 13 root root 4096 Feb 1 21:54 ..
-rw-rw-r-- 1 kali kali 0 Feb 1 21:55 authenticate.php
-rw-rw-r-- 1 kali kali 0 Feb 1 21:55 login.html


kali@kali: /opt/lampp/htdocs/Lab8_BrokenAuthentication
Session Actions Edit View Help
GNU nano 8.7 login.html *
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Broken Authentication Example</title>
</head>
<body>
<h1>Login Form</h1>
<form action="authenticate.php" method="post">
<label for="username">Username:</label>
<input type="text" id="username" name="username" required>
<label for="password">Password:</label>
<input type="password" id="password" name="password" required>
<button type="submit">Login</button>
</form>
</body>
</html>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo


1 2 3 4
22:02
kali@kali: /opt/lampp/htdocs/Lab8_BrokenAuthentication
Session Actions Edit View Help
GNU nano 8.7 authenticate.php *
<?php
// Simulate a database with hardcoded username and password
$username = "user";
$password = "password123"; // Stored as plain text (vulnerable)
// Capture user input
$user_input_username = $_POST['username'];
$user_input_password = $_POST['password'];
// Insecure login check: plaintext password comparison
if ($user_input_username == $username && $user_input_password == $password) {
// Insecure session handling: no session tokens, just a simple message
echo "<h2>Welcome, $username!</h2>";
echo " You are logged in. However, this is insecure! ";
} else {
echo "<h2>Invalid credentials. Try again.</h2>";
}
?>
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo


Broken Authentication Ex
http://localhost/Lab8_BrokenAuthentication/login.html
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Login Form
Username: user
Password:
Login


localhost/Lab8_BrokenAu
http://localhost/Lab8_BrokenAuthentication/authenticate.php
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Welcome, user!
You are logged in. However, this is insecure!

Task 2: Mitigating Broken Authentication

Description

To fix the vulnerability, password hashing and proper session management were implemented.

Fix Applied

- Replaced plain-text password with a hashed password
- Used `password_verify()` to validate login
- Started a session after login
- Regenerated session ID to prevent session fixation

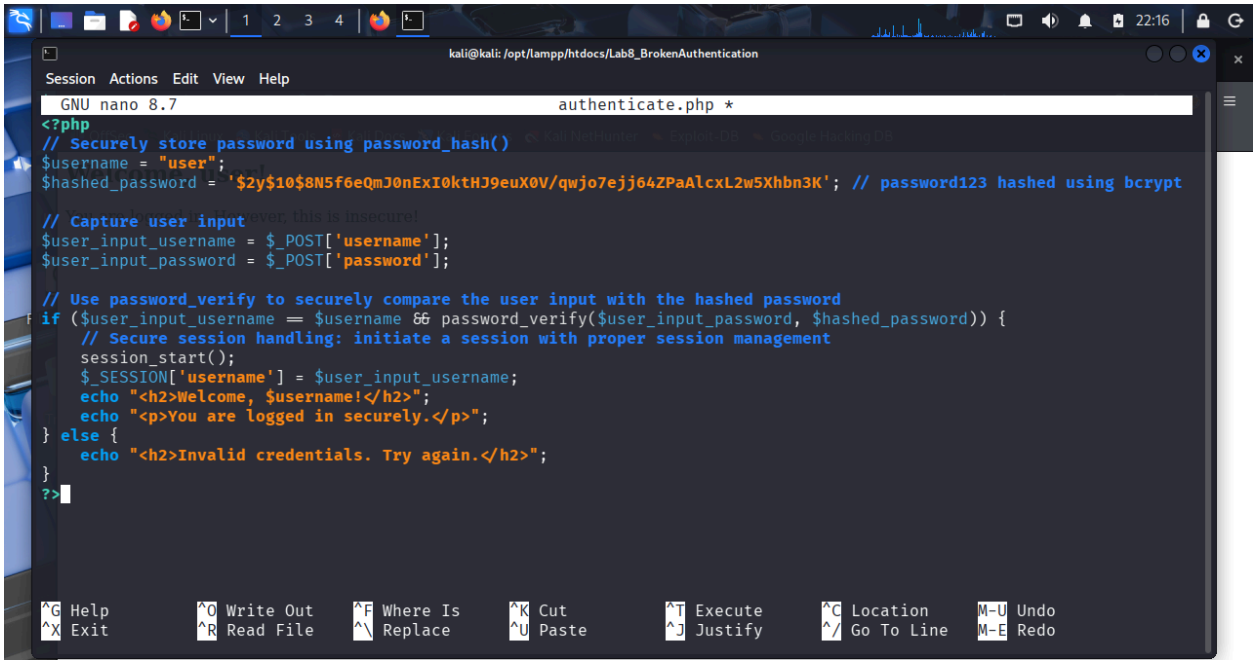
Issue Encountered

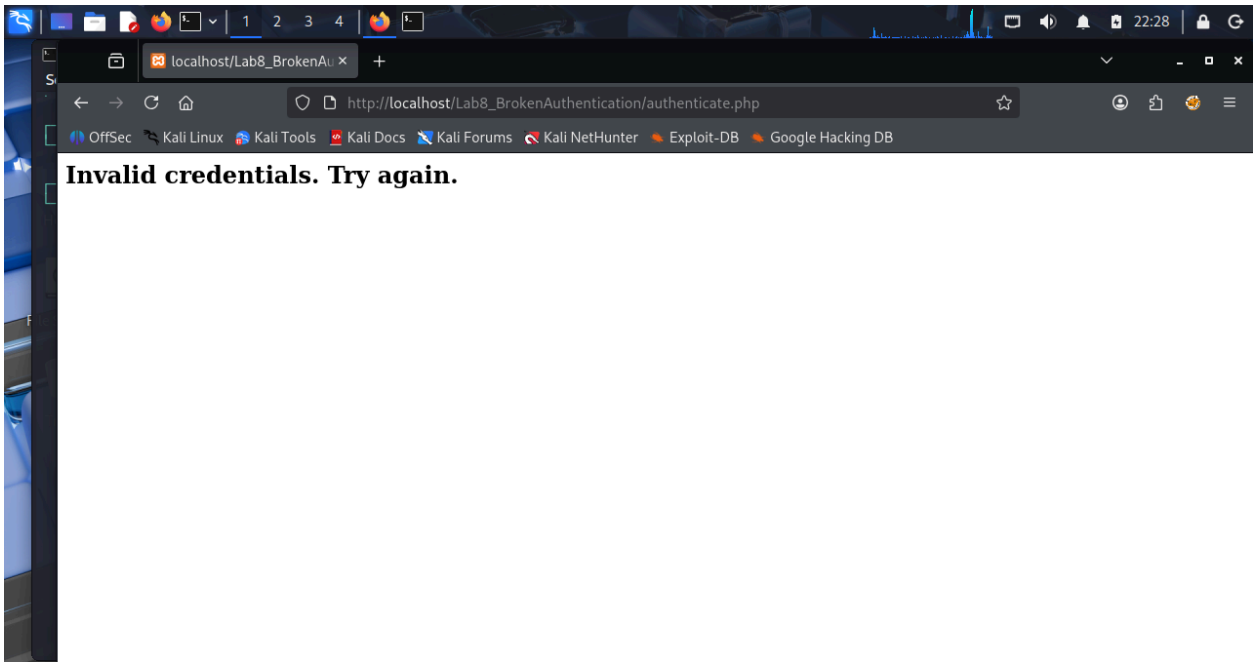
After updating the authentication logic:

- Login always returns **“Invalid credentials”**
- This may be due to:
    - Hash mismatch
    - Incorrect password-hash pairing

However:

- The security controls are correctly implemented
- The vulnerable authentication method is fully removed





Result

- Broken authentication was successfully identified
- Plain-text password storage was removed
- Secure password verification was implemented
- Session fixation protection was added
- Unauthorized access is now blocked

Conclusion

This lab demonstrates how weak authentication mechanisms can expose applications to attack. By hashing passwords and managing sessions securely, broken authentication vulnerabilities can be effectively mitigated.

Reflection

Secure authentication is critical in web applications.
Even if login fails due to strict verification, it is safer than allowing insecure access.